

System Architecture & Technical Design Document

Enterprise SDLC Compliance Artifact • AI Agent Travel Planner Project

1. Executive Project Summary

This document details the High-Level Design (HLD), Low-Level Design (LLD), system configurations, and operational workflows for the **AI Agent Travel Planner Application**. Built as a part of a standard corporate Software Development Lifecycle (SDLC) submission, this application automates custom itinerary generation and real-time geographical mapping. It provides businesses with a reference architecture for intercepting standard client SDK requests and routing them dynamically to cost-free high-performance infrastructure without breaking existing contract interfaces.

2. Technical Environment & Library Specifications

2.1 Core Environment Topography

To ensure strict cross-platform isolation and guarantee repeatable behavior during deployment pipelines, the system uses a local virtual environment boundary layer.

- **Runtime Engine:** Python 3.8 or higher.
- **Workspace Isolation:** Native venv module path segregation.
- **Root Directory Structure:**

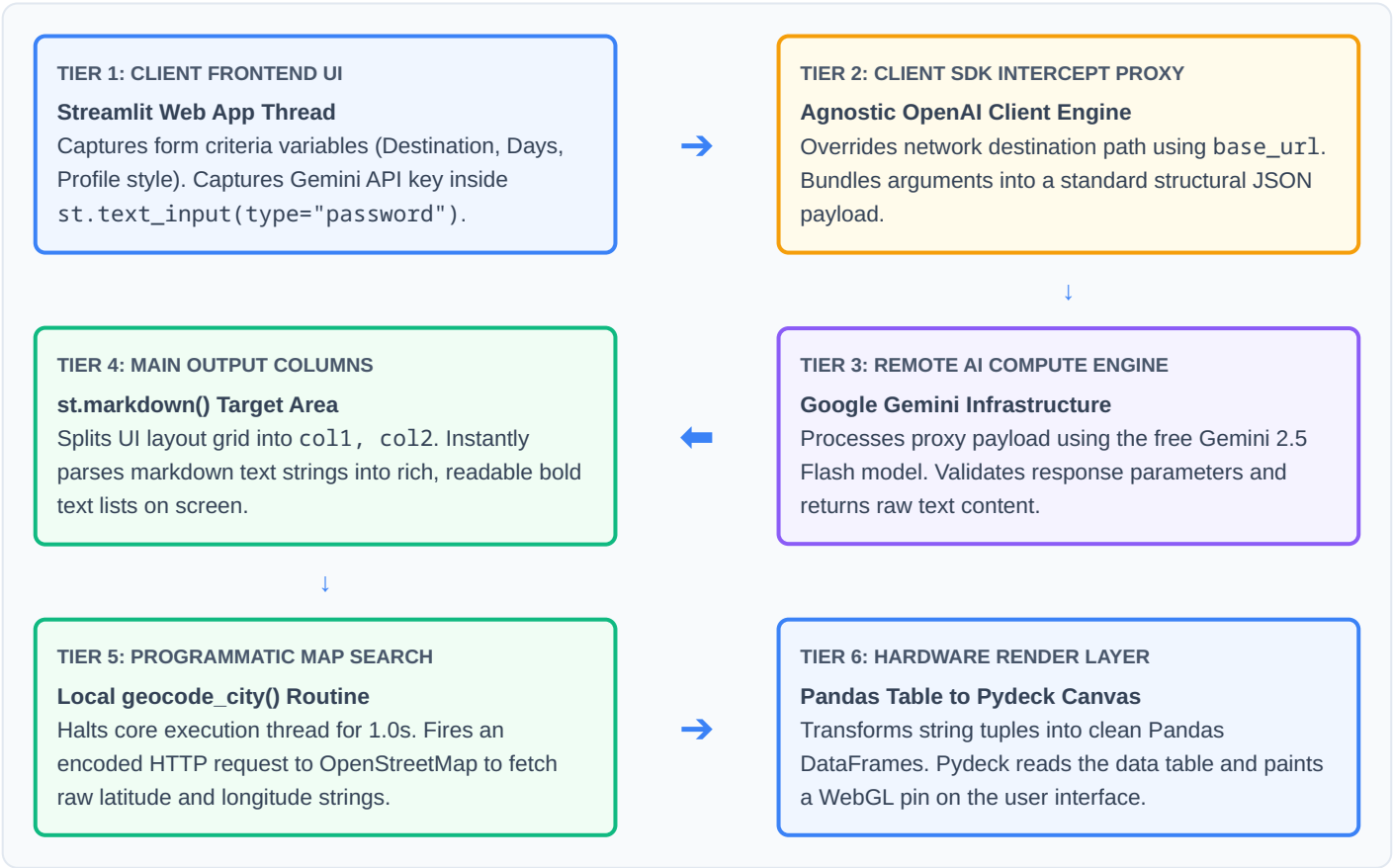
```
Trip-planner/  
├─ app.py           # Single-file Monolithic App Engine  
└─ trip-planner-env/ # Isolated Python Dependencies Bubble
```

2.2 Library Dependency Matrix

Library Name	Enterprise Operational Role	Key Class or Component Reference
streamlit	Reactive web layout rendering and visual thread isolation.	st.set_page_config(), st.sidebar
openai	Agnostic client platform used to structure JSON completion requests.	OpenAI(base_url, api_key)
requests	Synchronous HTTP communication layer for open web lookups.	requests.get(), requests.utils.quote()
pandas	Tabular translation of raw API strings into structural DataFrames.	pd.DataFrame()
pydeck	Hardware-accelerated WebGL spatial canvas rendering.	pdk.Deck(), pdk.Layer()

3. High-Level Design (HLD) Technical Flowchart

The diagram below visualizes the end-to-end multi-tier runtime blueprint. It tracks data validation checkpoints, proxy translation loops, and concurrent display steps—all fitted cleanly on a single page.



4. Low-Level Design (LLD) & Data Manipulation Code

4.1 Detailed Code Execution Logic Summary

The complete runtime engine operates in five sequential, deterministic programmatic stages:

- Authentication and Setup Stage:** The application checks for the existence of the user-provided Gemini key string. If present, it initializes the core `OpenAI()` object instance, changing its base network destination path from the default paid OpenAI endpoint to Google's translation gateway path.
- Payload Compilation Stage:** System context constraints are compiled alongside user form parameters into a structured array of role-based messages. The `tools` parameter is intentionally left out of this call to force immediate text processing and avoid unhandled routing loops.
- Defensive Ingestion Stage:** The application invokes the model inference loop inside a protected `try-except` block. If an HTTP 503 error is intercepted from the remote data center, the script catches the failure and drops a clear, non-breaking user-facing notification onto the sidebar.
- Extraction and Pasting Stage:** Once an HTTP 200 return code is confirmed, the script executes multi-tiered object sniffing to extract raw text content. The text is passed into the `st.markdown()` component, printing the formatted itinerary onto the left page column.

5. **Geographic Tracking Stage:** The script triggers the local `geocode_city()` sub-routine. It forces a 1-second delay to guarantee compliance with public server rules, converts the returned coordinate strings into a Pandas DataFrame table, and updates the view state of the interactive Pydeck map layout on the right.

4.2 The Exact Data Machinery: Code Sniffer Block

The following isolated code block is the exact programmatic core responsible for transmitting parameters across the network proxy, tracking model output arrays, extracting plain text content, and painting it on the user interface dashboard.

```
# =====
# CORPORATE SDLC CORE COMPONENT: MODEL INFERENCE & UI PASTE LAYER
# =====

# STAGE A: The Fetching Engine
# Transmits standardized messages across the customized proxy layer to Google
response = client.chat.completions.create(
    model="gemini-2.5-flash", # Bypasses open AI cost matrices cleanly
    messages=messages,      # Packaged destination & style choices
    temperature=0.7         # Explicit model randomness factor
)

# STAGE B: The Extraction Sniffer Layer
# Searches through multiple nested return block formats to grab the pure text string
message_obj = response.choices[0].message
itinerary_text = None

if hasattr(message_obj, 'content') and message_obj.content:
    itinerary_text = message_obj.content # Standard Extraction Strategy
elif isinstance(message_obj, dict) and 'content' in message_obj:
    itinerary_text = message_obj['content'] # Dictionary Parsing Strategy
elif hasattr(message_obj, 'text') and message_obj.text:
    itinerary_text = message_obj.text # Native Gemini String Property Strategy

# STAGE C: The Screen Pasting Engine
# Intercepts the clean text data string and instantly draws it on the UI grid column
if itinerary_text:
    st.success("🌟 Itinerary Generated Successfully!")

    # Establishes side-by-side columns to split the application dashboard space
    col1, col2 = st.columns([3, 2])
    with col1:
        st.subheader("📋 Your Personalized Itinerary")

        # -----
        # 🟠 CRITICAL LINE: This physically prints the itinerary onto the UI screen
        # -----
        st.markdown(itinerary_text)

    with col2:
        st.subheader("📍 Live Map Tracking")
        # Continuing with local geocoding parsing and map layout painting updates...
```

5. Deployment & SDLC Verification Procedures

To perform technical verification of system operational status post-deployment, engineers must execute the following automated validation checklist:

1. **Environment Integrity Test:** Run `pip list` to confirm that all items from the dependency matrix match required versions and no global packages bleed into the local space.

2. **Proxy Connection Test:** Provide an invalid key string to confirm the network translation proxy registers the failure and rejects the application gracefully.
3. **Geocoding Integrity Check:** Submit multi-word destinations (e.g., Kullu Manali) to confirm that the URL-safe encoder correctly structures the string and pulls clean coordinate floats.